Instalación, ejecución y pruebas del backend

TPI Logística de última milla · Backend de Aplicaciones 2026 · Comisión 3K4

Este manual lleva una computadora con **Windows** desde cero hasta el backend funcionando y probado con Postman. Sigue el mismo camino que se recorrió al ponerlo en marcha por primera vez, e incluye los errores que aparecieron en el camino y cómo se resolvieron.

Todos los comandos son para **PowerShell**. Donde dice `C:\Users\<usuario>\Documents\TPI Backend` va la carpeta donde se clonó el repositorio.

Parte	Contenido
1	Programas a instalar
2	Obtener el código
3	Base de datos
4	Keycloak (usuarios y tokens)
5	Compilar y levantar los servicios
6	Configuración: OSRM y mails
7	Probar los endpoints con Postman
8	Repetir la prueba (reset de datos)
9	Pruebas manuales: caídas, OSRM y mails
10	Errores frecuentes y cómo resolverlos
11	Resumen: arranque diario

1. Programas a instalar

Se instalan una sola vez. Después de cada instalación conviene **cerrar y volver a abrir PowerShell**, para que tome los cambios en el PATH.

Programa	Versión	Para qué
Git	cualquiera	Clonar el repositorio.
JDK (Java)	21 o superior	Compilar y ejecutar los servicios y Keycloak.
Maven	3.9 o superior	Compilar y levantar cada proyecto.
PostgreSQL	17 o superior (se usó la 18)	Las dos bases de datos.
Keycloak	26 (se usó la 26.4.0)	Login, tokens y roles.
Postman	app de escritorio	Probar los endpoints.
Postman CLI	opcional	Correr la colección desde la consola.

1.1 Git y Java

```
winget install Git.Git
winget install EclipseAdoptium.Temurin.21.JDK
```

Cómo saber que quedó bien: en una PowerShell nueva, `git --version` y `java -version` muestran sus versiones (Java 21 o más).

1.2 Maven

- Bajar el **Binary zip archive** desde https://maven.apache.org/download.cgi.
- Descomprimirlo, por ejemplo en `C:\apache-maven-3.9.x`.
- Agregar su carpeta `bin` al PATH: menú Inicio → buscar **variables de entorno** → **Editar las variables de entorno del sistema** → **Variables de entorno** → en *Variables de usuario* elegir **Path** → **Editar** → **Nuevo** → `C:\apache-maven-3.9.x\bin` → Aceptar.
- Si Maven se queja de `JAVA_HOME`, crear en esa misma ventana la variable de usuario `JAVA_HOME` con la carpeta del JDK (por ejemplo `C:\Program Files\Eclipse Adoptium\jdk-21...`).

Cómo saber que quedó bien: `mvn -version` muestra Maven 3.9 y la versión de Java.

1.3 PostgreSQL

- Bajar el instalador para Windows desde https://www.enterprisedb.com/downloads/postgres-postgresql-downloads.
- Ejecutarlo y aceptar los valores por defecto, con tres cuidados: en **Password** elegir una contraseña para el usuario `postgres` y **anotarla**; dejar el **Port** en `5432`; al final destildar **Stack Builder**.
- Queda instalado como servicio de Windows y arranca solo con la máquina.

> **Ojo.** `psql` no queda en el PATH. Se lo llama con la ruta completa, que incluye el número de versión: `C:\Program Files\PostgreSQL\18\bin\psql.exe`. Si se instaló otra versión, cambiar el 18.

1.4 Keycloak

No tiene instalador: es un .zip que se descomprime. En PowerShell:

```
cd $HOME
Invoke-WebRequest `
    https://github.com/keycloak/keycloak/releases/download/26.4.0/keycloak-26.4.0.zip `
    -OutFile keycloak.zip
Expand-Archive keycloak.zip -DestinationPath $HOME
Remove-Item keycloak.zip
```

Queda en `C:\Users\<usuario>\keycloak-26.4.0`. El realm del TP se copia en la parte 4.

1.5 Postman y Postman CLI

```
winget install Postman.Postman
```

La Postman CLI es opcional (permite correr la colección sin abrir la app). Instalador oficial para Windows, en una PowerShell:

```
[System.Net.ServicePointManager]::SecurityProtocol = 3072
iex ((New-Object System.Net.WebClient).DownloadString(
    'https://dl-cli.pstmn.io/install/win64.ps1'))
```

Cómo saber que quedó bien: `postman --version` muestra la versión de la CLI.

> **No confundir.** lo que se instala con `postman` desde la consola es la CLI, que no tiene interfaz gráfica. Para probar a mano hace falta la app de escritorio (`winget install Postman.Postman`).

2. Obtener el código

```
cd $HOME\Documents
git clone `
    https://github.com/SchallerFran/TP-Backend-Logistica-de-entregas-de-ultima-milla.git `
    "TPI Backend"
cd "TPI Backend"
```

Carpeta	Qué es
`logistica-commons`	Librería común (JAR) que usan los dos microservicios.
`pedidos-service`	Microservicio de pedidos y clientes (puerto 8081).
`despacho-service`	Microservicio de centros, flota, planificación y ejecución de rutas (puerto 8082).
`api-gateway`	Punto de entrada único (puerto 8080).
`infra`	Scripts de base de datos y el realm de Keycloak.
`postman`	Colección, entorno y guía de pruebas.

3. Base de datos (una sola vez)

Con PostgreSQL corriendo, desde la carpeta del repositorio:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -f infra/init-db.sql
```

Pide la contraseña de `postgres` (al escribirla no se ve nada; es normal). Crea las bases `pedidos_db` y `despacho_db` con sus usuarios. Las tablas las crea cada servicio al arrancar, y despacho-service carga solo los datos iniciales: 2 centros, 4 vehículos y 3 repartidores.

Cómo saber que quedó bien: la salida muestra varias líneas `CREATE ROLE`, `CREATE DATABASE` y `GRANT`.

Opcional, para no escribir la ruta completa las próximas veces: agregar `C:\Program Files\PostgreSQL\18\bin` al Path (mismo procedimiento que Maven, punto 1.2).

4. Keycloak

Primero se copia el realm del TP (roles, clients y usuarios de prueba) para que Keycloak lo importe al arrancar:

```
New-Item -ItemType Directory -Force $HOME\keycloak-26.4.0\data\import
Copy-Item infra\realm-logistica.json $HOME\keycloak-26.4.0\data\import\
```

Después se arranca **desde la carpeta de Keycloak**, en una terminal que queda abierta:

```
cd $HOME\keycloak-26.4.0
.\bin\kc.bat start-dev --http-port=8180 --import-realm
```

Cómo saber que quedó bien: aparece `Listening on: http://0.0.0.0:8180`, y `http://localhost:8180/realms/logistica` abre un JSON en el navegador.

Usuario	Contraseña	Rol
`cliente1`	`cliente1`	CLIENTE (comercio)
`despachante1`	`despachante1`	DESPACHANTE
`repartidor1`	`repartidor1`	REPARTIDOR (Juan Pérez, centro 1, turno 08 a 16)
`repartidor2`	`repartidor2`	REPARTIDOR (María Gómez, centro 1, turno 12 a 20)
`repartidor3`	`repartidor3`	REPARTIDOR (Lucas Sosa, centro 2, turno 08 a 16)
`admin1`	`admin1`	ADMIN

5. Compilar y levantar los servicios

5.1 Librería común (la primera vez y cada vez que cambie)

```
cd logistica-commons
mvn clean install
cd ..
```

Cómo saber que quedó bien: termina con `BUILD SUCCESS`.

5.2 Los tres servicios, en este orden

Cada uno en su propia terminal, que queda abierta. Esperar a que cada uno muestre `Started ...Application` antes de arrancar el siguiente:

```
# terminal 1
cd "<repo>\pedidos-service"  ;  mvn spring-boot:run

# terminal 2
cd "<repo>\despacho-service" ;  mvn spring-boot:run

# terminal 3
cd "<repo>\api-gateway"      ;  mvn spring-boot:run
```

También se pueden correr desde IntelliJ, con la clase `*Application` de cada proyecto.

Componente	Puerto	Comprobación en el navegador
Keycloak	8180	`http://localhost:8180/realms/logistica`
pedidos-service	8081	`http://localhost:8081/actuator/health` → UP · Swagger: `/swagger-ui.html`
despacho-service	8082	`http://localhost:8082/actuator/health` → UP · Swagger: `/swagger-ui.html`
api-gateway	8080	`http://localhost:8080/actuator/health` → UP

Tests automáticos (opcional): `mvn test` en `logistica-commons`, `pedidos-service` y `despacho-service`. Son 92 tests y no necesitan los servicios ni las bases levantados.

6. Configuración: OSRM y mails

Todas las credenciales y URLs están en los `application.yml` de cada servicio, así que **no hace falta configurar nada para que funcione**. Cualquier valor se puede sobrescribir con una variable de entorno.

6.1 Distancias por calle: OSRM

despacho-service calcula distancias y tiempos reales con **OSRM** (Open Source Routing Machine, datos de OpenStreetMap). Por defecto usa el servidor público `router.project-osrm.org`: no pide API key ni registro, pero admite una consulta por segundo y no garantiza disponibilidad. Si no responde, el sistema usa la caché o una estimación (línea recta × 1,3) y marca la ruta como `estimada`.

Para usar un OSRM propio (por ejemplo, el de la guía de la cátedra en Docker, puerto 5000):

```
setx OSRM_URL "http://localhost:5000"
```

> **Por qué no Google.** se eligió OSRM en lugar de Google Maps porque Google exige activar la facturación con tarjeta, aun para la cuota gratuita.

6.2 Mails de cambio de estado (Gmail)

pedidos-service avisa por mail al cliente cada cambio de estado de su pedido, desde `ultimamilla.tpbackend@gmail.com`. Gmail **no acepta la contraseña normal** de la cuenta para enviar desde una aplicación: hace falta una contraseña de aplicación.

- Iniciar sesión en Gmail con esa cuenta y activar la **verificación en 2 pasos** en https://myaccount.google.com/security.
- Entrar a https://myaccount.google.com/apppasswords, crear una contraseña llamada "TPI" y copiar los 16 caracteres.
- En `pedidos-service/src/main/resources/application.yml`, poner esa contraseña en `spring.mail.password` y reiniciar pedidos-service.

Si Gmail rechaza el login, el sistema sigue funcionando igual: solo queda el error en el log de pedidos-service.

7. Probar los endpoints con Postman

La carpeta `postman/` tiene una colección que prueba el sistema completo sola: pide los tokens, crea los datos, recorre el proceso de punta a punta y verifica cada respuesta con `pm.test`. Todas las requests van al gateway (`http://localhost:8080`), nunca directo a los microservicios.

7.1 Desde la app de Postman

- **Import** (arriba a la izquierda) y arrastrar los dos archivos de `postman/`: `TPI-Logistica.postman_collection.json` y `Local.postman_environment.json`.
- Arriba a la derecha, en el selector de entornos, elegir **Local**.
- Click derecho sobre la colección **TPI Logística** → **Run collection** → botón **Run TPI Logística**.
- Para ir paso a paso, abrir cada request en orden y apretar **Send**: cada una guarda en el entorno los ids y tokens que usan las siguientes.

7.2 Desde la consola (Postman CLI)

Desde la carpeta del repositorio:

```
postman collection run postman/TPI-Logistica.postman_collection.json `
    -e postman/Local.postman_environment.json
```

El aviso `No authorization data found. Please use the postman login command` se puede ignorar: solo indica que el resultado no se sube a la nube de Postman.

Resultado esperado: en la tabla final, `requests 53` y `assertions 91`, con **0** en la columna `failed`.

7.3 Qué prueba cada carpeta

Carpeta	Usuario	Qué hace y qué verifica
0. Auth	todos	Pide y guarda el token de cada usuario de prueba.
1. Datos maestros	`admin1`	Consulta centros, vehículos y repartidores; alta, modificación y baja de un vehículo.
2. Cliente	`cliente1`	Registra el perfil y crea 5 pedidos en el centro 1: urgente, normal, con frío, uno a cancelar y uno fuera de cobertura.
3. Planificación	`despachante1`	Planifica el día: el urgente y el normal comparten ruta, el de frío va en el vehículo refrigerado y el de fuera de cobertura queda NO_ASIGNABLE. Reprograma el no asignable. El cliente cancela un pedido asignado y la ruta se recalcula.
4. Ejecución	el repartidor de la ruta	Mi ruta, inicio (pedidos EN_CAMINO), una parada ENTREGADA y otra FALLIDA; la ruta queda FINALIZADA.
5. Seguimiento	varios	Pedido ENTREGADO; el fallido vuelve a RECIBIDO con 1 intento; historial completo; notificaciones; distancia con su fuente.
6. Negativos y seguridad	varios	Ventana invertida, centro inexistente, datos inválidos, cancelar en la calle, ruta ajena, sin token, rol sin permiso y /internal desde afuera: siempre 400, 401, 403, 409 o 422.

El detalle de qué regla verifica cada request está en `postman/Guia-pruebas-Postman.pdf` y en la matriz de trazabilidad del `README.md`.

8. Repetir la prueba (reset de datos)

Un vehículo y un repartidor hacen **una sola ruta por día** (regla RN11). Por eso, para volver a correr la colección el mismo día, primero hay que volver los datos al estado inicial. Desde la carpeta del repositorio:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -f infra/reset-datos.sql
```

Borra pedidos, clientes, rutas y avisos, y conserva los centros, vehículos y repartidores. No hace falta reiniciar los servicios.

Cómo saber que quedó bien: la salida muestra dos `TRUNCATE TABLE` y un `DELETE`.

9. Pruebas manuales

Estos escenarios requieren detener un servicio o revisar un buzón, así que no están en la colección.

9.1 Tolerancia a caídas: el repartidor sigue trabajando con pedidos-service caído

#	Acción	Resultado esperado
1	Reset (parte 8) y correr la colección paso a paso hasta **P3 Iniciar ruta** (carpeta 4).	Ruta EN_CURSO.
2	Detener pedidos-service (Ctrl+C en su terminal).	-
3	Ejecutar **P4 Parada ENTREGADA**.	**200**: despacho registra la entrega igual.
4	Ejecutar **Notificaciones con error** (carpeta 5) cambiando `estado=ERROR` por `estado=PENDIENTE`.	Aparece el aviso pendiente.
5	Volver a levantar pedidos-service.	En menos de 15 s el aviso queda ENVIADA y el pedido pasa a ENTREGADO.

9.2 Alta de pedido con despacho-service caído

Detener despacho-service y ejecutar **P1 Crear pedido NORMAL**: responde **503** y el pedido no se crea, porque no se puede validar el centro.

9.3 Respaldo de distancias

Levantar despacho-service con `OSRM_URL` apuntando a una dirección que no existe (por ejemplo, en esa terminal: `$env:OSRM_URL = "http://localhost:9"` antes de `mvn spring-boot:run`). La planificación se completa igual, `fuentesDistancia` muestra `ESTIMADA` y las rutas salen con `estimada = true`.

9.4 Mails

Con la contraseña de aplicación cargada (parte 6.2), registrar el perfil del cliente con un email real y correr la colección: llega un mail por el alta y por cada cambio de estado. En el log de pedidos-service aparece `Mail enviado a ...` por cada uno.

10. Errores frecuentes y cómo resolverlos

Todos estos aparecieron al poner en marcha el sistema por primera vez.

Error	Causa	Solución
`bin\kc.bat : El módulo 'bin' no pudo cargarse`	Se ejecutó desde la carpeta del TP y sin `.\`.	Ir a la carpeta de Keycloak y usar `.\bin\kc.bat`. Revisar que el parámetro sea `--import-realm` (con m).
`psql : El término 'psql' no se reconoce`	psql no está en el PATH.	Usar la ruta completa con `&` adelante (parte 3).
`...\PostgreSQL\17\bin\psql.exe no se reconoce`	Está instalada otra versión.	Cambiar el número por la versión instalada (por ejemplo 18).
Una variable de entorno nueva no se ve	`setx` solo afecta a las terminales que se abren después.	Cerrar y volver a abrir la terminal o IntelliJ, y reiniciar el servicio.

Error	Causa	Solución
Falla la carpeta 0 de Postman	Keycloak no está corriendo o no importó el realm.	Revisar que `realm-logistica.json` esté en `data\import` y arrancar con `--import-realm`.
`ECONNREFUSED` a 8080	El gateway no está corriendo.	Levantarlo (parte 5.2).
Falla desde **P2 Planificar el día**	La colección ya se corrió hoy: los vehículos y repartidores están usados.	Reset de datos (parte 8).
`401` en requests sueltas	Los tokens duran 15 minutos.	Volver a ejecutar la carpeta 0.
`503`	Un microservicio no responde.	Levantarlo y repetir.
Las distancias salen `ESTIMADA`	OSRM no respondió (el servidor público limita a 1 consulta por segundo).	Esperar unos segundos y repetir; la caché evita volver a consultar los mismos pares.
`No se pudo enviar` en el log de pedidos	Gmail rechazó la contraseña.	Usar una contraseña de aplicación (parte 6.2).

11. Resumen: arranque diario

Con todo ya instalado, cada vez que se quiera usar el sistema:

#	Dónde	Comando
1	PostgreSQL	Ya arranca solo con Windows.
2	Terminal 1	`cd $HOME\keycloak-26.4.0 ; .\bin\kc.bat start-dev --http-port=8180 --import-realm`
3	Terminal 2	`cd <repo>\pedidos-service ; mvn spring-boot:run`
4	Terminal 3	`cd <repo>\despacho-service ; mvn spring-boot:run`
5	Terminal 4	`cd <repo>\api-gateway ; mvn spring-boot:run`
6	Terminal 5	Si ya se probó hoy: reset (parte 8). Después, correr la colección (parte 7).